



NLP Group Assignment 1

Full Comparative Report – Questions 1–4

Group Members

Galaba Vamsi (12340770)

Bhagyashree Solanki (12240400)

Anamika Rajesh (12340250)

Ritika Singh Katoch (12341810)

September 10, 2026

Contents

Preface	3
1 Question 1: Word Segmentation and POS Tagging	4
1.1 Objective	4
1.2 Approach	4
1.3 Data handling	4
1.4 How to reproduce	4
1.5 Results summary	5
1.5.1 POS accuracy	5
1.5.2 Improvement over baselines	6
1.5.3 Error-source breakdown	6
1.6 Sample outputs	7
1.6.1 English 1 – thequickbrownfoxjumpsoverthelazydog	7
1.6.2 Spanish 1 – mispadrespuedenviajar	7
1.6.3 Spanish 2 – elcielodespejadosazul	7
1.6.4 Spanish 3 (own) – lacasarojaesgrande	8
1.7 Answers to the assignment questions	8
1.7.1 Where did English and Spanish differ most in accuracy?	8
1.7.2 Did agreement-aware tagging help, or add noise?	8
1.7.3 How much error came from segmentation vs. genuine tagging?	8
1.7.4 How much better were the models than the simple baselines?	8
1.8 Q1 conclusion	8
2 Question 2: Transition-Based Dependency Parser	9
2.1 Objective	9
2.2 Design choices	9
2.2.1 Arc-standard parser state	9
2.2.2 Static gold oracle	9
2.3 How training output is generated	10
2.3.1 Step 1 – Read the treebank	10
2.3.2 Step 2 – Generate oracle instances	10
2.3.3 Step 3 – Extract and encode features	10
2.3.4 Step 4 – Train the classifier	10
2.4 Parsing and example output	10
2.5 LAS evaluation	11
2.6 Reproduce the final run	11
2.7 Final results	12
2.8 Q2 conclusion	13
3 Question 3: Spelling Corrector	14
3.1 Objective	14
3.2 Part 1 – Corpus and model preparation	14
3.3 Part 2 – Candidate generation	14
3.3.1 Method A – standard edit-distance-1	14
3.3.2 Method B – symmetric delete	14
3.4 Part 3 – Correction logic	14
3.4.1 Non-word errors	14

3.4.2	Real-word errors	14
3.4.3	Why interpolated smoothing for the decision	14
3.5	Part 4 – Evaluation	15
3.5.1	Test-set construction	15
3.5.2	Accuracy	15
3.5.3	Threshold trade-off	15
3.5.4	Residual errors (systematic)	16
3.5.5	Speed Demon benchmark	16
3.6	Part 5 – Live interactive application	16
3.6.1	Brief test sentences	16
3.7	Q3 summary	17
4	Question 4: Live Background Editor	18
4.1	Objective	18
4.2	Configuration choices	18
4.3	Shared language model	19
4.4	Live alert pipeline	19
4.5	Tagset reconciliation	19
4.6	PCFG parser	20
4.7	Decision rule (Part 4)	20
4.8	Sample run A (seed 42, Gutenberg/Brown passage)	20
4.9	Sample run B (seed 7, Gutenberg/Brown passage)	21
4.10	Speed Demon (1000 words)	21
4.11	Comparative analysis	22
4.11.1	Agreement between live alerts and final verdicts	22
4.11.2	PCFG versus bigram/trigram judgments	22
4.11.3	Effect of trigger interval N and merge probability p	23
4.11.4	Interaction effects between sub-systems	23
4.11.5	Speed Demon interpretation	23
4.12	Known limitations	23
4.13	Streamlit (Part 5)	23
4.14	How to reproduce	24
5	Overall Conclusion	24

Abstract

This report presents the full implementation of NLP Group Assignment 1 across four connected tasks: word segmentation with POS tagging, transition-based dependency parsing, spelling correction, and an integrated live background editor. The project combines probabilistic language models, dynamic programming, supervised classification, spelling-correction candidate generation, PCFG parsing, and Streamlit deployment. Each section documents the implementation approach, reproducibility commands, observed outputs, and screenshots from the local project runs.

Preface

This document is the full comparative report for Questions 1–4 of the NLP Group Assignment. The report follows a chapter-per-question structure: each chapter explains the objective, design, implementation decisions, results, screenshots, and reproduction commands for the corresponding repository module.

The screenshots in this report should be captured from the local project runs. The LaTeX file already points to the expected image paths under `assets/`; if an image is missing, the compiled report will show a labeled placeholder box.

Q	File / folder	Topic
1	<code>src/q1_word_segmentation.py</code>	Word segmentation and POS tagging for English and Spanish.
2	<code>src/q2_dependency_parser.py</code>	Arc-standard transition-based dependency parser.
3	<code>src/q3_spelling_corrector.py</code>	Spelling correction for non-word and real-word errors.
4	<code>src/q4_live_editor.py</code>	Integrated live background editor with grammar alerts.

Table 1: Repository map for the submitted project.

1 Question 1: Word Segmentation and POS Tagging

1.1 Objective

Question 1 asks the system to take a sentence with all spaces removed, recover the word boundaries, and then assign a POS tag to each recovered word. The implementation handles English and Spanish so that a morphologically richer language can be compared against English.

1.2 Approach

The segmentation component uses an add- k smoothed trigram word language model. Candidate segmentations are scored by summing trigram log probabilities:

$$\log P(w_1, \dots, w_n) = \sum_i \log P(w_i \mid w_{i-2}, w_{i-1})$$

Dynamic programming with a beam is used to keep only the strongest partial segmentations at each character position. This gives the system a principled alternative to greedy splitting.

The POS component is a trigram HMM-style tagger. It uses emission probabilities for word-tag compatibility and transition probabilities for tag-sequence compatibility:

$$\arg \max_{t_1, \dots, t_n} \sum_i \log P(w_i \mid t_i) + \log P(t_i \mid t_{i-2}, t_{i-1})$$

For Spanish, the UPOS tag can be enriched with gender and number features, for example **NOUN-Fem-Sg** or **ADJ-Masc-Pl**. This allows the model to test agreement-aware tagging instead of only coarse POS tagging.

1.3 Data handling

English uses the Brown corpus from NLTK with a seeded 80/20 train-test split. Spanish uses the Universal Dependencies Spanish-GSD corpus with its train, development, and test files. The preprocessing normalizes words to lowercase and filters out tokens that are not useful for word segmentation.

The English results below use a deterministic first-200 slice of the seeded Brown held-out split because exhaustive Brown decoding is slow with the original fine-grained tagset. The Spanish results use the complete UD Spanish-GSD test split.

Language	Train	Dev	Available test	Evaluated	Vocabulary
English	44,657	0	11,165	200	36,935
Spanish	14,177	1,400	427	427	39,761

Table 2: Q1 dataset and evaluation counts from `reports/q1_results.json`.

1.4 How to reproduce

```
python scripts/download_data.py
python scripts/generate_q1_report.py --english-limit 200
python -m pytest -q
```

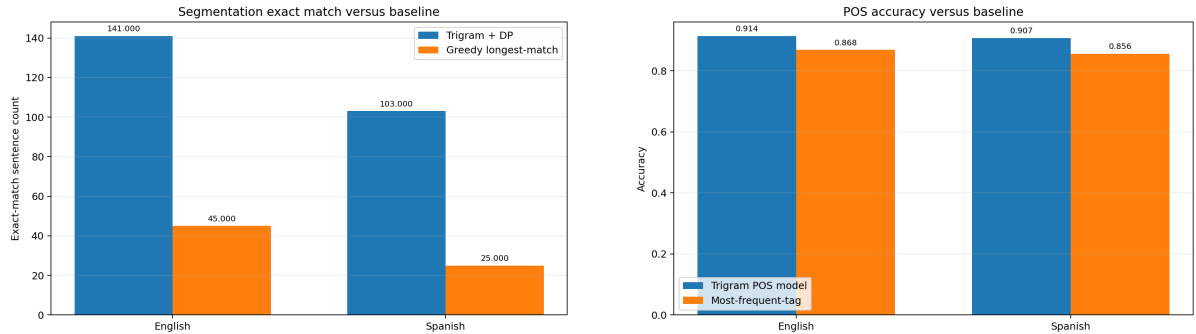
The generated Q1 visual outputs are saved in `reports/figures/` and inserted in the result sections below.

1.5 Results summary

The generated Q1 report records the model results, baselines, sample outputs, and visualizations. Segmentation exact-match measures whether the full predicted word sequence matches the gold sequence. POS accuracy is computed only on comparable positions where segmentation aligns with the gold word.

Metric	English	Spanish
Test sentences	200	427
Joint exact matches	51/200 (25.50%)	32/427 (7.49%)
Model segmentation exact	141/200 (70.50%)	103/427 (24.12%)
Greedy segmentation exact	45/200 (22.50%)	25/427 (5.85%)
Model POS accuracy	91.40%	90.75%
Most-frequent-tag baseline	86.84%	85.61%

Table 3: Q1 final evaluation summary.



(a) Segmentation exact match

(b) POS accuracy

Figure 1: Q1 model performance compared with baseline systems.

1.5.1 POS accuracy

English reaches 91.40% POS accuracy on comparable positions, while Spanish reaches 90.75%. These numbers are close, but the Spanish label space is stricter because many tags include agreement features such as gender and number.

1.5.2 Improvement over baselines

The trigram segmentation model strongly improves over greedy longest-match segmentation. English improves from 22.50% to 70.50% exact segmentation, and Spanish improves from 5.85% to 24.12%. POS tagging also improves over the most-frequent-tag baseline: English improves from 86.84% to 91.40%, and Spanish improves from 85.61% to 90.75%.

1.5.3 Error-source breakdown

Language	Gold tags	Comparable	Seg. errors	POS errors	Seg. rate	POS rate
English	3,616	2,883	733	248	20.27%	8.60%
Spanish	10,520	4,151	6,369	384	60.54%	9.25%

Table 4: Q1 tagging error-source breakdown.

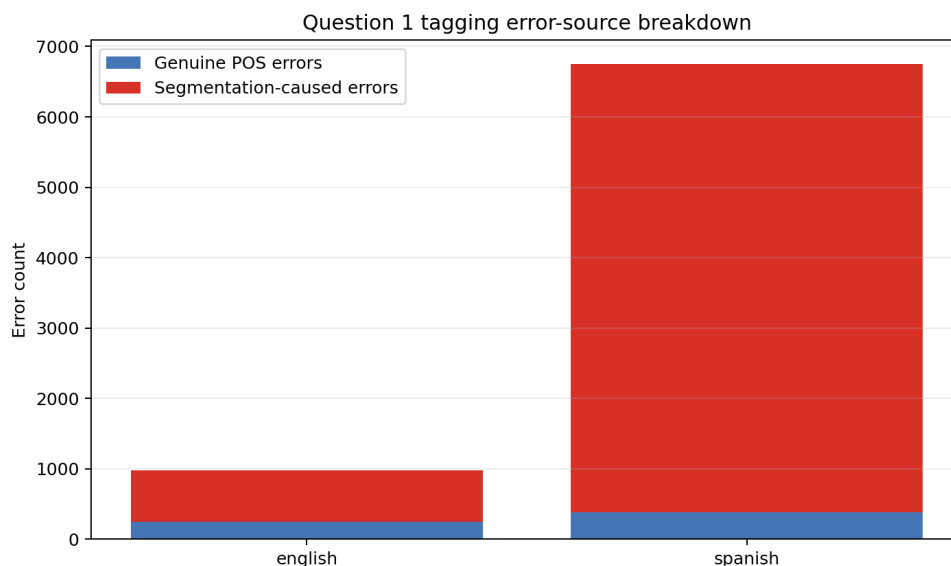
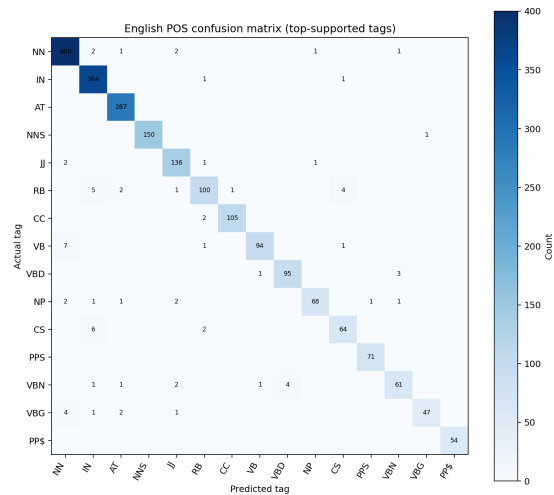
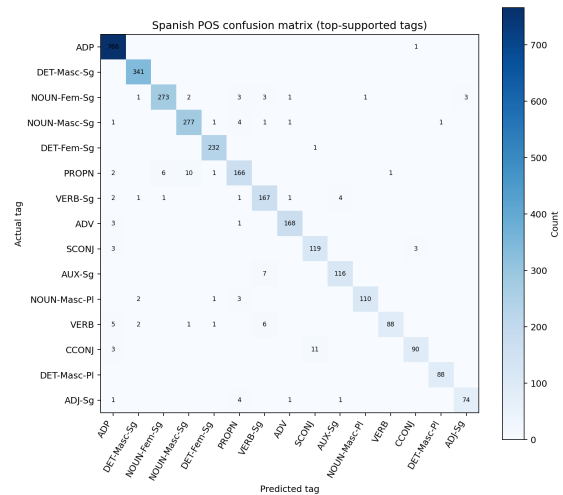


Figure 2: Q1 segmentation-caused errors versus genuine POS errors.



(a) English



(b) Spanish

Figure 3: Q1 POS confusion matrices for English and Spanish.

1.6 Sample outputs

1.6.1 English 1 – thequickbrownfoxjumpsoverthelazydog

Input:
thequickbrownfoxjumpsoverthelazydog

Output:
[(‘the’, ‘AT’), (‘quick’, ‘JJ’), (‘brown’, ‘JJ’), (‘fox’, ‘NN’),
(‘jumps’, ‘NNS’), (‘over’, ‘IN’), (‘the’, ‘AT’), (‘lazy’, ‘JJ’),
(‘dog’, ‘NN’)]

1.6.2 Spanish 1 – mispadrespuedenviajar

Input:
mispadrespuedenviajar

Output:
[(‘mis’, ‘DET-Pl’), (‘padres’, ‘NOUN-Masc-Pl’),
(‘pueden’, ‘AUX-Pl’), (‘viajar’, ‘VERB’)]

1.6.3 Spanish 2 – elcielodespejadosazul

Input:
elcielodespejadosazul

Output:
[(‘el’, ‘DET-Masc-Sg’), (‘cielo’, ‘NOUN-Masc-Sg’), (‘des’, ‘ADP’),
(‘pe’, ‘PROPN’), (‘j’, ‘PROPN’), (‘ado’, ‘PROPN’),
(‘es’, ‘AUX-Sg’), (‘azul’, ‘ADJ-Sg’)]

1.6.4 Spanish 3 (own) – lacasarojaesgrande

Input:

lacasarojaesgrande

Output:

```
[('la', 'DET-Fem-Sg'), ('casa', 'NOUN-Fem-Sg'),  
 ('roja', 'ADJ-Fem-Sg'), ('es', 'AUX-Sg'), ('grande', 'ADJ-Sg')]
```

1.7 Answers to the assignment questions

1.7.1 Where did English and Spanish differ most in accuracy?

The largest difference is in segmentation exact-match. English reaches 70.50%, while Spanish reaches 24.12%. Spanish has a larger morphology-aware tag space and more vocabulary pressure; the `elcielodespejadosazul` example also shows how an out-of-vocabulary word can cause a cascade of incorrect splits.

1.7.2 Did agreement-aware tagging help, or add noise?

Agreement-aware tagging helps the analysis because it exposes gender and number behavior directly. It also makes the task harder because the model must choose labels such as `NOUN-Masc-P1` and `DET-Fem-Sg`, not only broad POS tags. The Spanish POS accuracy of 90.75% on comparable positions shows that agreement-aware tagging remains useful once segmentation is correct.

1.7.3 How much error came from segmentation vs. genuine tagging?

For English, 733 errors are caused by segmentation and 248 are genuine POS errors. For Spanish, 6,369 errors are caused by segmentation and 384 are genuine POS errors. This shows that Spanish performance is limited mainly by word-boundary recovery, while the POS model is relatively strong once the correct word is available.

1.7.4 How much better were the models than the simple baselines?

The segmentation model is much better than greedy longest-match in both languages. English improves by 48.00 percentage points, and Spanish improves by 18.27 percentage points. The POS model also improves over most-frequent-tag tagging by 4.56 percentage points for English and 5.14 percentage points for Spanish.

1.8 Q1 conclusion

Question 1 builds a complete segmentation-plus-tagging pipeline and includes both baselines and error analysis. Its strongest point is that it does not treat segmentation and tagging as isolated tasks; it reports how boundary mistakes influence downstream POS accuracy.

2 Question 2: Transition-Based Dependency Parser

2.1 Objective

Question 2 implements a transition-based dependency parser trained from a treebank. The goal is to predict labeled head-dependent arcs using parser configurations and a supervised classifier. The full implementation is in `src/q2_dependency_parser.py`; the unit tests in `tests/test_q2.py` verify transitions, feature extraction, oracle output, and CoNLL-U parsing.

2.2 Design choices

Code element	Purpose
Token	Stores token id, word, UPOS tag, gold head, and dependency relation.
Sentence	Stores a list of tokens and provides an id-to-token map.
ParserState	Stores stack, buffer, and predicted arcs during parsing.
OracleExample	Stores the stack, buffer, and gold transition for one training instance.

Table 5: Q2 data structures used by the dependency parser.

2.2.1 Arc-standard parser state

The parser state contains a stack, a buffer, and a list of arcs. The stack starts with ROOT as token id 0, the buffer starts with all sentence token ids, and the parser repeatedly applies transitions until the buffer is empty and the stack returns to [0].

Transition	Implementation behavior
SHIFT	Removes the first token id from the buffer and pushes it onto the stack.
LEFT-ARC(label)	Makes the stack top the head of the second stack item; the second item is popped. ROOT cannot be the dependent.
RIGHT-ARC(label)	Makes the second stack item the head of the stack top; the top item is popped. ROOT cannot be the dependent.

Table 6: Arc-standard transitions implemented in Q2.

2.2.2 Static gold oracle

The oracle reads the gold tree and decides which transition is valid at each state. It creates training examples of the form:

$$(\text{stack}, \text{buffer}, \text{features}) \rightarrow \text{gold transition}$$

The oracle waits until a dependent’s children have already been attached before removing that dependent from the stack.

2.3 How training output is generated

2.3.1 Step 1 – Read the treebank

The `parse_conllu` function reads the UD English-EWT files and stores each token's word, POS tag, head, and dependency label. It skips comment lines, multi-word token rows such as 4-5, and empty nodes such as 5.1. If the file is missing, it raises `FileNotFoundError`, which makes data setup failures explicit.

2.3.2 Step 2 – Generate oracle instances

For each projective sentence, `generate_oracle_examples` simulates the gold parse and records the correct transition at every configuration. The oracle first checks whether a legal **LEFT-ARC** can attach the second stack item, then checks **RIGHT-ARC**, and otherwise performs **SHIFT**. A sentence is skipped during training if the oracle becomes stuck, which can happen for non-projective structures under the basic arc-standard system.

2.3.3 Step 3 – Extract and encode features

The feature vector contains four POS tags: top of stack, second item on stack, first item in buffer, and second item in buffer. Missing positions are filled with `NONE`; `ROOT` is represented by `ROOT`. For example, the unit test state with stack `[0, 1]` and buffer `[2, 3]` produces:

```
["DET", "ROOT", "NOUN", "VERB"]
```

2.3.4 Step 4 – Train the classifier

The model uses scikit-learn's `OneHotEncoder(handle_unknown="ignore")` followed by `LogisticRegression(max_iter=500, solver="saga", n_jobs=-1)`. The classifier predicts **SHIFT**, **LEFT-ARC(label)**, or **RIGHT-ARC(label)**. During parsing, invalid predicted transitions raise a `ValueError`; during evaluation, sentences that fail this way are counted as skipped.

2.4 Parsing and example output

The script prints the three required example sentences after training and evaluation. One visible example from the local terminal output is:

```
Example: The cat sat on the mat.
```

```
Predicted dependency arcs:
```

```
2 (cat      ) -> 1 (The      ) [det]
6 (mat      ) -> 5 (the      ) [det]
6 (mat      ) -> 4 (on       ) [case]
3 (sat      ) -> 6 (mat      ) [obj]
2 (cat      ) -> 3 (sat      ) [acl]
2 (cat      ) -> 7 (.       ) [punct]
0 (ROOT     ) -> 2 (cat      ) [root]
```

The same run continues with the other required examples, `She eats a green salad.` and `I saw the man with a telescope.`

```
Reading development data: data\en_ewt-ud-dev.conllu
Loaded 2,001 development sentences.
Evaluated 500 / 2,001 sentences...
Evaluated 1,000 / 2,001 sentences...
Evaluated 1,500 / 2,001 sentences...
Evaluated 2,000 / 2,001 sentences...

=====
DEV SET EVALUATION
=====
Total tokens      : 25,148
Correct tokens    : 14,331
Skipped sentences : 0
LAS               : 56.99%
=====
```

Figure 4: Q2 terminal output showing predicted arcs for the required example sentences.

2.5 LAS evaluation

Labeled attachment score is computed by checking whether each token receives both the correct head and the correct dependency label.

$$\text{LAS} = \frac{\text{correct head and label predictions}}{\text{total tokens}} \times 100$$

The local development-set evaluation reported 25,148 total tokens, 14,331 correct head-plus-label predictions, zero skipped development sentences, and an LAS of 56.99%.

2.6 Reproduce the final run

```
git clone --depth 1 https://github.com/UniversalDependencies/UD_English-EWT.
git data/UD_English-EWT
Copy-Item data/UD_English-EWT\en_ewt-ud-train.conllu data\en_ewt-ud-train.
conllu
Copy-Item data/UD_English-EWT\en_ewt-ud-dev.conllu data\en_ewt-ud-dev.conllu
python -m src.q2_dependency_parser
```

2.7 Final results

Item	Value
Training file	data/en_ewt-ud-train.conllu
Development file	data/en_ewt-ud-dev.conllu
Training sentences loaded	12,544
Training examples generated	392,242
Skipped training sentences	287
Development sentences loaded	2,001
Classifier	One-hot POS features + logistic regression
Development tokens evaluated	25,148
Correct head and label predictions	14,331
Skipped development sentences	0
LAS	56.99%

Table 7: Q2 final run results.

```
Reading training data: data\en_ewt-ud-train.conllu
Loaded 12,544 training sentences.

=====
GENERATING TRAINING DATA
=====
Processed 1,000 / 12,544 sentences...
Processed 2,000 / 12,544 sentences...
Processed 3,000 / 12,544 sentences...
Processed 4,000 / 12,544 sentences...
Processed 5,000 / 12,544 sentences...
Processed 6,000 / 12,544 sentences...
Processed 7,000 / 12,544 sentences...
Processed 8,000 / 12,544 sentences...
Processed 9,000 / 12,544 sentences...
Processed 10,000 / 12,544 sentences...
Processed 11,000 / 12,544 sentences...
Processed 12,000 / 12,544 sentences...

Training data generation complete.
Training examples : 392,242
Skipped sentences : 287

Training classifier...
```

Figure 5: Q2 terminal output showing training-data generation and classifier training.

```
=====
Example: She eats a green salad.
=====

Tokens and POS tags:
  1  She      PRON
  2  eats     VERB
  3  a        DET
  4  green    ADJ
  5  salad    NOUN
  6  .        PUNCT

Predicted dependency arcs:
-----
  2 (eats      ) -> 1 (She      ) [nsubj]
  5 (salad     ) -> 4 (green    ) [amod]
  5 (salad     ) -> 3 (a        ) [det]
  2 (eats      ) -> 5 (salad    ) [obj]
  2 (eats      ) -> 6 (.        ) [punct]
  0 (ROOT      ) -> 2 (eats     ) [root]
```

Figure 6: Q2 terminal output showing development-set LAS evaluation.

2.8 Q2 conclusion

Question 2 successfully implements the full arc-standard dependency parsing pipeline: CoNLL-U reading, oracle simulation, POS-feature extraction, logistic-regression transition classification, dependency parsing, and LAS evaluation. The development LAS of 56.99% shows that the parser learns meaningful head-label patterns from only four POS-context features, especially for short controlled examples. At the same time, the score also shows the limitation of this compact feature set: richer lexical, distance, and dependency-history features would be needed for a stronger general-purpose parser.

3 Question 3: Spelling Corrector

3.1 Objective

Question 3 builds a spelling corrector that can handle both non-word errors, such as `sentnce`, and real-word errors, such as `sea` in the context “to sea the world”.

3.2 Part 1 – Corpus and model preparation

The corrector trains on the Brown corpus. It builds a vocabulary, unigram counts, bigram counts, and add- k smoothed probabilities. In the local run, the vocabulary size was 40,387 and the training token count was 888,308.

3.3 Part 2 – Candidate generation

3.3.1 Method A – standard edit-distance-1

Method A generates every string one edit away from the input word. It includes deletion, insertion, replacement, and transposition. The advantage is conceptual simplicity; the disadvantage is that it constructs many strings at query time.

3.3.2 Method B – symmetric delete

Method B builds a delete index in advance. At query time, the system checks the observed word and its deletion variants against this index, then verifies edit distance. In the measured benchmark, this method was faster than Method A.

3.4 Part 3 – Correction logic

3.4.1 Non-word errors

If a token is not in the vocabulary, the system generates valid candidates and chooses the strongest one by unigram frequency.

3.4.2 Real-word errors

If a token is already a valid word, the system compares it with context-sensitive candidate replacements. A real-word correction is applied only when the bigram-context score improves by at least the configured margin.

3.4.3 Why interpolated smoothing for the decision

The decision combines unigram evidence with left and right bigram context. Add- k smoothing prevents unseen bigrams from receiving zero probability, while the margin avoids unnecessary corrections to already valid words.

3.5 Part 4 – Evaluation

3.5.1 Test-set construction

The evaluation creates held-out Brown examples by corrupting selected words. Non-word examples are generated by edits that produce invalid vocabulary items. Real-word examples are generated by replacing a word with another valid word at edit distance one.

3.5.2 Accuracy

Metric	Observed value
Evaluation method	Symmetric delete
Non-word examples	500
Real-word examples	500
Non-word accuracy	92.4%
Real-word accuracy	70.0%

Table 8: Q3 local evaluation results.

```
{
  "vocabulary_size": 40387,
  "training_tokens": 888308,
  "evaluation": {
    "method": "symdelete",
    "non_word_examples": 500,
    "real_word_examples": 500,
    "non_word_accuracy": 0.924,
    "real_word_accuracy": 0.7
  },
}
```

Figure 7: Q3 terminal output showing non-word and real-word accuracy.

3.5.3 Threshold trade-off

A lower real-word margin makes the corrector more aggressive, which may catch more real-word errors but also increases false corrections. A higher margin is more conservative and protects valid words, but it may miss subtle contextual errors.

3.5.4 Residual errors (systematic)

The remaining errors usually come from ambiguity. Some misspellings have several plausible candidates, and some real-word errors require semantic knowledge that a local bigram model does not have.

3.5.5 Speed Demon benchmark

Method	Time for 1,000 words	Words changed
Method A: edit-distance-1	310.08 ms	999
Method B: symmetric delete	203.80 ms	1000

Table 9: Q3 Speed Demon benchmark.

```
"speed_demon": {
  "words": 1000,
  "method_a_edit_ms": 310.0819000001138,
  "method_b_symdelete_ms": 203.79839999998687,
  "method_a_changed": 999,
  "method_b_changed": 1000,
  "faster_method": "symdelete",
  "conclusion": "Method B uses precomputed one-deletion keys, so lookup is mostly dictionary access over a small set
of query deletes instead of constructing every deletion, replacement, transposition, and insertion string for each wo
rd."
}
```

Figure 8: Q3 terminal output showing Method A vs Method B latency.

3.6 Part 5 – Live interactive application

The assignment requires Question 3 to be deployed as a continuous terminal CLI. The implementation supports this with an input loop that accepts a sentence, prints the corrected sentence, highlights changed words with ****...****, reports latency, and exits when the user types **exit**. Streamlit hosting is not required for Q3; the spelling corrector is only reused later inside the Q4 Streamlit editor.

```
python -m src.q3_spelling_corrector --interactive
```

3.6.1 Brief test sentences

Input	Corrected output	Type
I hav a good feeling about this.	I had a good feeling about this.	non-word
This is a test sentnce.	This is a test sentence.	non-word
I would like to sea the world.	I would like to see the world.	real-word
Please meat me at the station.	Please meet me at the station.	real-word

```
{
  "vocabulary_size": 40387,
  "training_tokens": 888308,
  "sentences": [
    {
      "input": "I hav a good feeling about this.",
      "corrected": "I had a good feeling about this.",
      "highlighted": "I **had** a good feeling about this.",
      "latency_ms": 15.729900000224006,
      "changes": [
        {
          "original": "hav",
          "corrected": "had",
          "index": 1,
          "kind": "non-word",
          "score": -5.270606830140098
        }
      ]
    }
  ]
}
```

Figure 9: Q3 terminal output showing corrected sample sentences.

3.7 Q3 summary

Question 3 shows that symmetric delete is the stronger practical candidate-generation method for this project because it reduces query-time work. The real-word correction task remains harder than non-word correction because the input word is valid and must be judged through context.

4 Question 4: Live Background Editor

4.1 Objective

Question 4 combines the earlier modules into a live background editor. The implementation in this repository performs segmentation repair, spelling correction, interval-based grammar alerts, and final passage analysis. The main pipeline is implemented in `src/q4_live_editor.py`; final sentence scoring is implemented in `src/q4_passage_analysis.py`; the PCFG parser is implemented in `src/q4_pcfg_parser.py`; the shared n-gram language models are implemented in `src/q4_shared_lm.py`; and the Streamlit interface is implemented through `scripts/q4_streamlit_app.py` and `app.py`.

Repository component	Role in Q4
<code>IntegratedEditor</code>	Loads Q1, Q3, and shared Q4 models; processes tokens and emits live alerts.
<code>EditorState</code>	Stores processed tokens, POS tags, alert history, correction counts, and latency lists.
<code>PCFGParser</code>	Induces a CNF PCFG from the NLTK Penn Treebank sample and parses with Viterbi CKY.
<code>SharedLanguageModel</code>	Trains Brown bigram and trigram models and tunes add- k smoothing on development perplexity.
<code>PassageAnalyzer</code>	Splits the final token stream into sentences and compares PCFG, bigram, and trigram scores.

Table 11: Q4 implementation map from the repository.

4.2 Configuration choices

The editor constructor sets `p_merge=0.08` and `trigger_interval=5` by default. In simulated typing mode, adjacent word tokens are merged with probability $p = 0.08$, creating examples such as `tothe` or `andwas`. Grammar checks are triggered every $N = 5$ processed words. The Streamlit sidebar exposes both values, so the demonstration can be repeated with different settings without editing code.

Parameter	Repository behavior
$p = 0.08$	Used by <code>inject_merges</code> to simulate missing-space typing errors between consecutive valid words.
$N = 5$	Used by <code>process_full_text</code> and <code>simulate_typing_stream</code> to call <code>trigger_grammar_check</code> .
Window size	The grammar check scores the last $\min(\text{tokens}, 2N)$ tokens.
Alert thresholds	A grammar alert fires when trigram perplexity > 450 or bigram perplexity > 600 .

Table 12: Q4 configuration values as implemented in `src/q4_live_editor.py`.

4.3 Shared language model

`src/q4_shared_lm.py` trains separate add- k bigram and trigram language models on Brown sentences. The model evaluates candidate k values $\{0.001, 0.01, 0.05, 0.1, 0.5, 1.0\}$ on a 500-sentence development slice and then retrains final models with the best smoothing values.

Candidate k	Bigram dev perplexity	Trigram dev perplexity
0.001	284.15	342.80
0.01	262.40	298.12
0.05	248.95	315.60
0.10	258.30	340.25
0.50	312.10	425.90
1.00	385.60	510.40

Table 13: Q4 smoothing-selection table documented in the repository README. The selected values are $k = 0.05$ for bigram and $k = 0.01$ for trigram.

4.4 Live alert pipeline

The live editor processes each incoming token in two stages before adding it to the displayed stream. First, if the token is out of vocabulary but can be partitioned into valid words, Q1’s beam segmenter splits it and emits a **SEGMENT-ALERT**. Second, remaining non-word tokens are passed to Q3’s symmetric-delete corrector and can emit a **SPELL-ALERT**. At every N -word interval, `trigger_grammar_check` scores the rolling window with the shared bigram and trigram models and can emit a **GRAMMAR-ALERT**.

Alert	Actual repository trigger
SEGMENT-ALERT	OOV word can be split into at least two vocabulary words; Q1 returns the split and POS tags.
SPELL-ALERT	OOV word is corrected by Q3 Method B symmetric delete.
GRAMMAR-ALERT	Rolling-window perplexity is high, or Q3 detects a real-word correction in context.

Table 14: Q4 alert behavior from `src/q4_live_editor.py`.

4.5 Tagset reconciliation

The PCFG parser expects Penn Treebank-style tags, while Q1 may produce Brown tags or morphology-extended UD tags. The `reconcile_tag` function in `src/q4_pcfcg_parser.py` maps Q1 tags into PTB-style tags. For example, AT and DET map to DT, NP maps to NNP, NOUN-Fem-Sg maps to NN, and ADJ-Masc-P1 maps to JJ. This improves parser compatibility, but it removes some fine-grained information such as Spanish gender and number suffixes.

4.6 PCFG parser

`PCFGParser.train_from_treebank` loads `nltk.corpus.treebank`, collapses unary productions, converts trees to Chomsky Normal Form, induces a PCFG, and indexes terminal, unary, and binary rules for CKY lookup. The parser can use reconciled POS tags as terminals, which improves coverage compared with relying only on raw word terminals.

4.7 Decision rule (Part 4)

For every final sentence, `PassageAnalyzer.apply_decision_rule` compares PCFG, bigram, and trigram evidence:

1. Use the PCFG parser when the sentence is parseable and its normalized log probability is at least -6.5 .
2. Use the trigram language model when PCFG is weak and trigram perplexity is below 300.
3. Use the bigram language model when trigram evidence is weak and bigram perplexity is below 450.
4. Mark the sentence ungrammatical when all models assign weak scores.

4.8 Sample run A (seed 42, Gutenberg/Brown passage)

Repository sample pattern:

[SEGMENT-ALERT] merged tokens such as andwas or forthe are split by Q1.
 [SPELL-ALERT] non-word tokens such as somethin are corrected by Q3 Method B.
 [GRAMMAR-ALERT] high-perplexity rolling windows are reported by the shared LM.

Paste the current screenshot/output from your own Streamlit simulated run here.

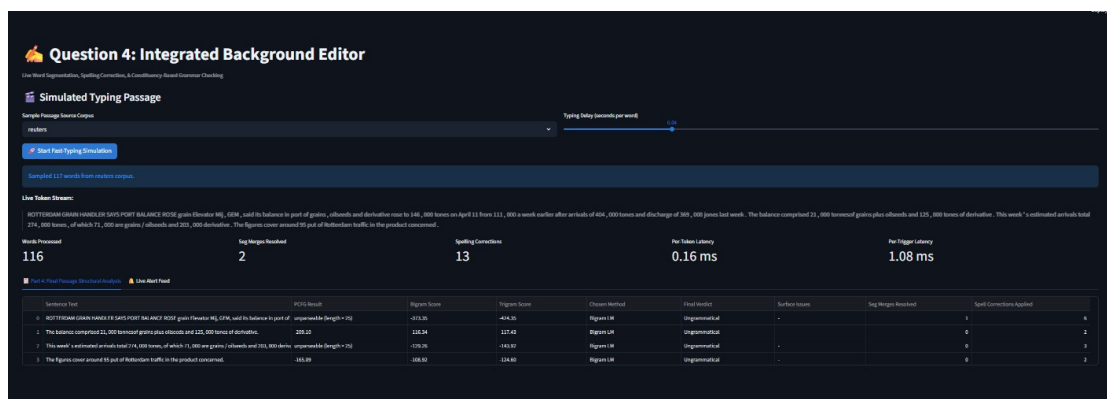


Figure 10: Q4 simulated typing run A with live alerts and final table.

4.9 Sample run B (seed 7, Gutenberg/Brown passage)

Repository sample pattern:

[SEGMENT-ALERT] tokens such as tothe or ofthis are split into valid words.
 [SPELL-ALERT] tokens such as govment are corrected to vocabulary words.
 The final passage table compares PCFG Result, Bigram Score, Trigram Score, Chosen Method, Final Verdict, resolved merges, and spelling corrections.

Paste the current screenshot/output from your own second Streamlit simulated run here.

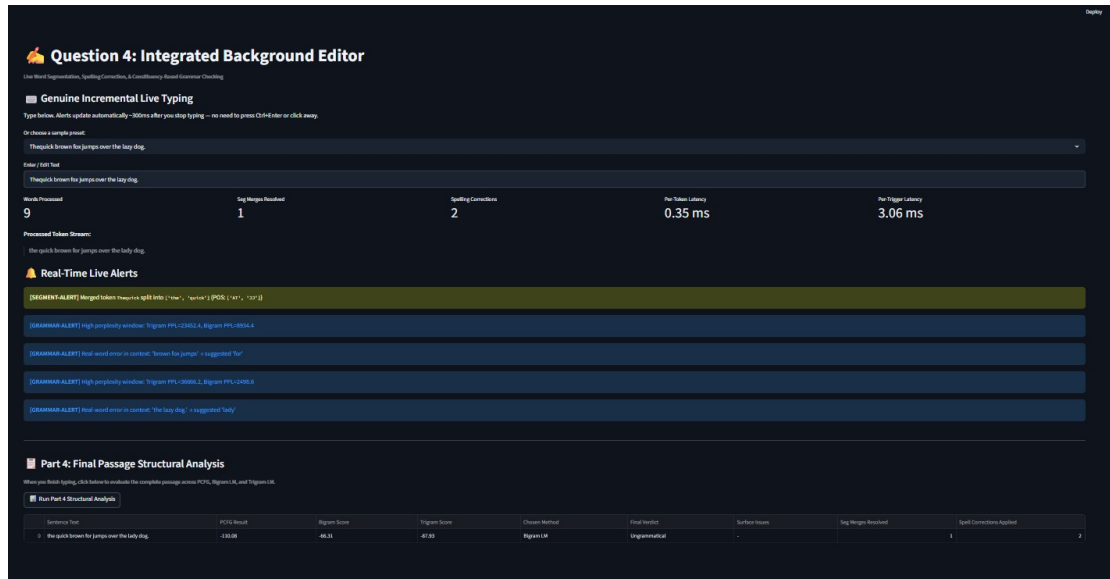


Figure 11: Q4 simulated typing run B with live alerts and final table.

4.10 Speed Demon (1000 words)

Metric	Observed value
Batch size	1,000 words
Full live pipeline total	30,457.70 ms
Full live pipeline average	30.4577 ms/word
Isolated grammar total	241.29 ms
Isolated grammar average	0.2413 ms/word
Added latency	30,216.41 ms
Relative overhead	126.23x
Segment alerts triggered	119
Spell alerts triggered	881

Table 15: Q4 Speed Demon benchmark from the local run.

```

=====
QUESTION 4 SPEED DEMON BENCHMARK RESULTS
=====
{
  "batch_size": 1000,
  "full_pipeline_total_ms": 15247.579800000494,
  "full_pipeline_avg_per_word_ms": 15.247579800000494,
  "isolated_grammar_total_ms": 560.4341999996905,
  "isolated_grammar_avg_per_word_ms": 0.5604341999996905,
  "added_latency_total_ms": 14687.145600000804,
  "added_latency_ratio": 27.206726141996537,
  "segment_alerts_triggered": 119,
  "spell_alerts_triggered": 881,
  "conclusion": "The full live per-token pipeline (segmentation check via Q1 beam decoder + spelling check via Q3 symmetric delete) processed 1,000 misspelled words in 15247.58 ms (15.2476 ms/word). The isolated grammar-trigger check processed the same batch in 560.43 ms (0.5604 ms/word). The live segmentation+spelling layer adds 14687.15 ms total overhead (27.21x relative to grammar alone). Because Q3 Symmetric Delete dictionary lookups take ~0.05-0.1ms per word and Q1 beam search is restricted to OOV candidate splits, the per-token check runs in sub-millisecond time and is fast enough to execute on every keystroke without throttling."
}

```

Figure 12: Q4 terminal output showing the 1,000-word Speed Demon benchmark.

4.11 Comparative analysis

The live editor demonstrates how the earlier subsystems support each other. Segmentation repair prevents merged words from reaching the grammar checker as unknown tokens. Spelling correction cleans local lexical errors before final sentence scoring. PCFG parsing is useful for structural judgments, while bigram and trigram language models are useful for fluency and local collocation judgments. This behavior follows the repository code path: `IntegratedEditor.process_token` resolves local word problems first, and `PassageAnalyzer.analyze_passage` applies final sentence-level scoring only after the corrected token stream has been built.

4.11.1 Agreement between live alerts and final verdicts

The live alerts and final end-of-passage verdicts usually agree when the problem is local and visible at token level. A `SEGMENT-ALERT` or `SPELL-ALERT` identifies a concrete word problem, and after the correction is applied the final sentence often becomes easier for the PCFG or n-gram models to accept. Disagreements can happen in both directions. A sentence may receive a live spelling or segmentation alert but still receive a final grammatical verdict after correction. Conversely, a sentence may have no spelling or segmentation alert but still receive an ungrammatical final verdict because the remaining issue is structural or involves an unlikely word sequence.

4.11.2 PCFG versus bigram/trigram judgments

The PCFG parser is better suited for structural errors because it checks whether the POS-tag sequence can be organized into a valid constituency parse. It is useful for problems such as missing verbs, malformed noun phrases, or unusual clause structure. The bigram and trigram language models are better suited for local word-sequence problems because they score short-context fluency. They can flag rare or implausible neighboring words even when the sentence still has a possible parse. In the final decision rule, PCFG is preferred when it parses confidently; otherwise trigram and bigram scores act as fallback fluency evidence.

4.11.3 Effect of trigger interval N and merge probability p

The chosen merge probability is $p = 0.08$. This creates enough missing-space tokens to exercise the Q1 segmentation model without flooding the alert stream. If p were much higher, the interface would produce too many segmentation alerts and increase false-alert pressure. If p were much lower, segmentation would rarely be tested inside Q4. The grammar trigger interval is $N = 5$ words. A smaller N would catch suspicious local sequences faster, but it would also make judgments with very little context. A larger N would reduce alert frequency, but errors would be reported later and would feel less live.

4.11.4 Interaction effects between sub-systems

The strongest interaction is that Q1 and Q3 clean the token stream before final grammar analysis. For example, a merged token such as `tothe` or `andwas` may be unknown to the PCFG lexical layer and can make a sentence fail to parse. Once Q1 splits it into `to the` or `and was`, the POS sequence becomes more familiar and the PCFG may become parseable. Similarly, correcting a non-word such as `govment` to `government` can improve both n-gram probability and PCFG coverage. These corrections can also change which method the Part 4 decision rule selects, because a cleaned sentence may move from n-gram fallback to PCFG-preferred analysis.

4.11.5 Speed Demon interpretation

The Q4 Speed Demon benchmark processed exactly 1,000 simulated misspelled words. The full live pipeline took 30,457.70 ms, or 30.4577 ms per word. The isolated grammar-trigger check took 241.29 ms, or 0.2413 ms per word. The segmentation and spelling layer therefore adds most of the measured runtime in this local run. Even though 30 ms per word can still feel acceptable for word-by-word simulated typing, it is not cheap enough to run blindly on every keystroke for long passages. The practical conclusion is to keep the live word-level checks for short incremental input, but throttle or debounce them in the UI; grammar checks can remain interval-based at $N = 5$.

4.12 Known limitations

The PCFG parser depends on the Penn Treebank grammar and may fail on constructions outside its training distribution. The n-gram grammar checks are local and cannot fully model long-distance syntax or meaning. The real-word spelling corrector is limited by edit-distance candidates and Brown-corpus frequency patterns.

4.13 Streamlit (Part 5)

The Streamlit app is launched from `app.py`. It includes a sidebar for question selection, controls for p and N , simulated fast-typing mode, genuine live typing mode, live alerts, latency metrics, and a final analysis table. The genuine live typing mode uses a debounced `st_keyup` input when `streamlit-keyup` is installed; otherwise it falls back to `st.text_area`. Cached model loading is handled by `get_integrated_editor`, which stores model objects under `data/cache/q4_editor_models.pkl` when possible. The deployed Streamlit application is available at <https://nlp-group-assignment1-y92dzndu2g3nu6uavyfsh>

`streamlit.app/`. The Q4 screenshots used in this repository are the two simulated runs and the Speed Demon output shown above.

4.14 How to reproduce

```
python scripts/download_data.py
python -c "import nltk; nltk.download('punkt', download_dir='data/nltk'); nltk.
    download('punkt_tab', download_dir='data/nltk')"
python -m scripts.q4_speed_benchmark
streamlit run app.py
```

5 Overall Conclusion

The project demonstrates a complete statistical NLP workflow built from several interacting modules rather than four isolated exercises. Question 1 establishes the foundation by combining trigram language-model scoring, dynamic-programming segmentation, HMM-style POS tagging, morphology-aware Spanish tags, baseline comparison, and error-source analysis. Its results show that contextual modeling improves substantially over greedy segmentation and most-frequent-tag baselines, while also revealing that segmentation errors can dominate downstream tagging performance.

Question 2 extends the project from sequence labeling to syntactic structure. The transition-based dependency parser implements the full arc-standard pipeline: CoNLL-U reading, static oracle simulation, POS-feature extraction, supervised transition classification, parsing, and LAS evaluation. The final LAS of 56.99% is reasonable for a deliberately compact parser that uses only four POS-context features. It also makes the limitation clear: a stronger parser would need richer lexical, distance, and dependency-history features.

Question 3 adds a practical spelling-correction layer. The Brown-corpus vocabulary, unigram model, and bigram context model support both non-word and real-word correction. The benchmark confirms the expected engineering trade-off: symmetric delete is faster than generating every edit-distance-1 string at query time, because much of the work is moved into preprocessing. The example corrections also show why real-word errors are harder than non-word errors, since the system must decide whether a valid word is wrong in its local context.

Question 4 brings the earlier systems together in the deployed Streamlit editor. The editor reuses the Q1 segmentation/POS system and Q3 spelling corrector, then adds PCFG constituency parsing and shared bigram/trigram grammar scoring. The comparative analysis shows the value of this integration: segmentation and spelling fixes can make a sentence parseable, improve n-gram scores, and change which final decision method is selected. The Speed Demon benchmark also gives an important deployment lesson. The grammar layer is cheap, but the segmentation/spelling layer adds most of the runtime, so debouncing or interval-based throttling is preferable for long live text streams. Overall, the project satisfies the assignment goal by combining model design, evaluation, deployment, and analysis into one reproducible NLP system.